for a `Venue` object. Ninja's decorators support naming in a fashion similar to a `path` object in the urls.py file. The only complication is they are namespaced. If your `NinjaAPI` object has no `version` argument, the namespace is `api-1.0.0:`. If your `NinjaAPI` object does have a `version` argument (like in our code), the namespace starts with `api-`, then appends the value of `version`, and finishes with a colon (`:`). In either case, the name you give the API endpoint comes after the `:`. The following listing contains the code for serializing a `Venue` object and the API endpoint for fetching a single instance.

Listing 11.7 API endpoint for fetching a `Venue` object

```
# RiffMates/bands/api.py
from django.shortcuts import get_object_or_404
from django.urls import reverse
from django.utils.text import slugify

from ninja import Router, ModelSchema

from bands.models import Venue

router = Router()

class VenueOut(ModelSchema):
    slug: str
    url: str

    class Meta:
        model = Venue
        fields = ["id", "name", "description"]

    @staticmethod
    def resolve_slug(obj):
        slug = slugify(obj.name) + "-" + str(obj.id)
        return slug

    @staticmethod
    def resolve_url(obj):
        url = reverse("api-1.0:fetch_venue", args=[obj.id, ])
        return url

@router.get("/venue/{venue_id}/",
    response=VenueOut,
    url_name="fetch_venue")
def fetch_venue(request, venue_id):
    venue = get_object_or_404(Venue, id=venue_id)
    return venue
```

As this is a new api.py file, it needs its own Router.

Additional fields get declared in a Schema or ModelSchema class, using Python type hinting.

Serialize the id, name, and description fields from the Venue object.

Ninja uses .resolve_X() static methods to populate the corresponding field. The .resolve_slug() method uses Django's slugify() function to create a slug based on the Venue object's name and ID.

The .resolve_url() method looks up the API endpoint named fetch_venue and returns the corresponding URL.

The url_name argument to a Ninja decorator declares a name for the endpoint that can be looked up with a call to reverse().

In addition to adding the code in listing 11.7, don't forget to register the new `router` with the `NinjaAPI` object in urls.py, using `bands` as the router's base name. Once done, you can run the development server and visit http://localhost:8000/api/v1/bands/venue/1/. If you followed along way back in chapter 4, you will get the following result: